

1. Linear Probing vs Separate Chaining

Feature	Linear Probing	Separate Chaining
Collision handling	Searches next empty slot sequentially.	Stores collided elements in linked list.
Clustering	Suffers from primary clustering.	No primary clustering.
Memory usage	Uses the hash table itself; no extra pointers.	Requires extra memory for linked list.
Search	Average: $O(1)$, Worst: $O(n)$	Average: $O(1)$, Worst: $O(n)$
Insertion	Average: $O(1)$, Worst: $O(n)$	Average: $O(1)$, Worst: $O(n)$
Deletion	More complicated, tomb-stone may be needed.	Easier
High load factor	Performance decreases rapidly.	Handles high load factor better.
Advantages	Simple, cache-friendly, low overhead.	Flexible and handles collisions efficiently.
Limitations	Clustering & performance degradation.	Extra memory & pointer overhead.
Real-world use	Small/medium hash tables with good cache performance.	Large-scale hash tables and systems with variable data.

CONCLUSION:

Separate chaining is generally more suitable for large scale applications, because it can handle higher load factor and collisions more effectively.

2. Quick Sort vs Merge Sort for 10 Million linked-list Node

Feature	Quick Sort	Merge Sort
Average time	$O(n \log n)$	$O(n \log n)$
Worst-case time	$O(n^2)$	$O(n \log n)$
Space for linked list	$O(\log n)$ average recursion stack	$O(\log n)$ recursion stack; merging can be done without array storage.
In-place	Yes, generally.	Not traditionally for array, but suitable.
Cache behavior	Good for array because of locality.	Sequential access is good for linked list.
Stability	Usually not stable	Stable
Large linked list	Less suitable	Highly suitable
Array	Usually very efficient	Requires $O(n)$ auxiliary space in typical.

For linked list

→ linked list do not support efficient random access.

→ Merge sort works well.

→ Merging (LL) can perform changing pointers.

→ It guarantees $O(\log n)$.

→ It does not require additional array of 10 million elements.

For array

→ Quick sort is often preferred in practice because it can in-place and has good cache locality.

3. Quadratic Probing vs Double Hashing

Feature	Quadratic Probing	Double Hashing
Collision handling	Uses quadratic increments.	Uses a second hash function.
Probe sequence	$h(k), h(k)+1^2, h(k)+2^2, \dots$	$h_1(k) + 1 \times h_2(k)$
Primary clustering	Can occur	Greatly reduced
Collision efficiency	Moderate	High
Performance	Good at moderate load factors.	Better for higher load factors
Memory	$O(m)$	$O(m)$
Dense hash tables	Less suitable when very dense	More suitable
Complexity	Average $O(1)$, worst $O(n)$	Average $O(1)$, worst $O(m)$

CONCLUSION:

Double hashing is generally better because it produces a more variable probe sequence & reduces clustering.

Quadratic probing is simpler & reduces primary clustering compared with linear probing, but secondary clustering still occurs.

H. Best-case vs Worst-case Time Complexity

Algorithm	Best Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$

CONCLUSION:

Best case shows maximum time, worst case shows max time. $O(n \log n)$ performance.

I. Recursive vs Iterative Quick Sort

Feature	Recursive	Iterative
Method	Uses recursive	Uses explicit stack
Average Time	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n^2)$	$O(n^2)$
Stack	$O(\log n)$ average	$O(\log n)$
Advantage	simple implement	Better stack ctrl
Limitation	Stack overflow risk	More complex

CONCLUSION:

Recursive Quick Sort is simpler, while Iterative Quick Sort provides better ctrl over stack memory.